JMAC Sprint 2 Design Rationale

May McGrath 33880417
Joyce Leung 32531885
Ahn Nguyen 32528558
Claire Zhang 33111332

Gen AI Declaration

The team acknowledges using Microsoft Copilot (https://copilot.microsoft.com/) to assist in the placeholder creation of optional God Card use. This LLM was prompted to

- create a JavaSwing YES/NO user prompt method that would return a boolean value. The code generated was a JOptionPanel which was in a brief iteration used as a placeholder for testing that GodCard use could be optional. This code was later removed and the logic rethought by the team to instead cancel GodCard usage on worker click to better fit with the pre-existing code.
- Recommend design patterns to use for a java implementation of the board game Santorini, which was then rewritten in the team's own words.
- Code a scaffold for a ClickHandler class and edited to add working logic.
- Write dot points into concise paragraphs for design pattern rationale and reworded in own words.

Video Demonstration

This is in mp4 format and can be accessed via the following link: Panopto Link

Diagrams

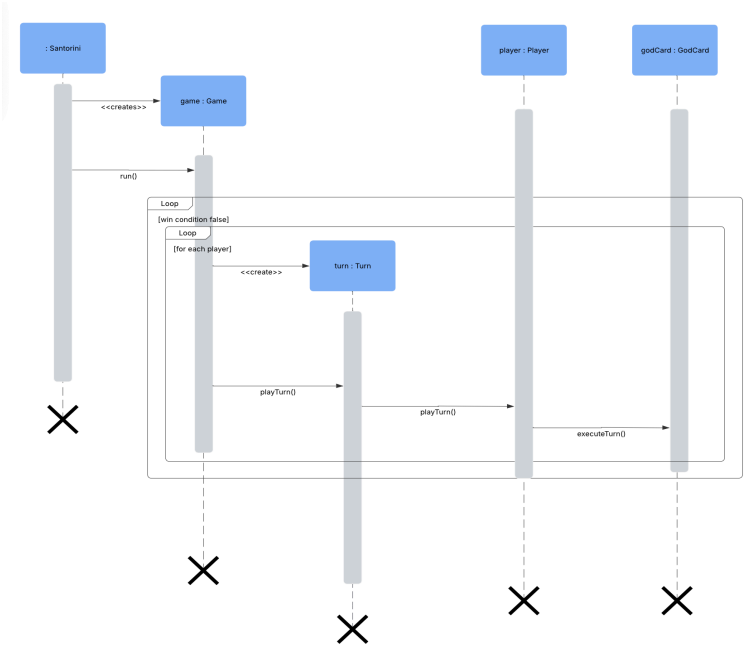
The UML Diagram is available on the team git here: UML Diagram, as are the Sequence Diagrams here. In these links, earlier iterations of the final diagrams included here can be accessed as well.

Sequence Diagrams

Turn

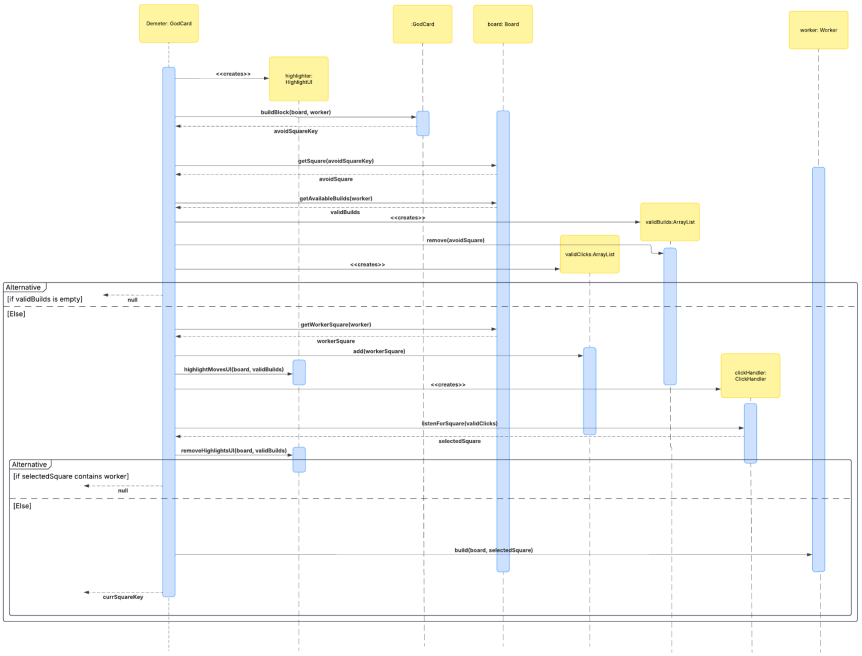
This sequence diagram shows the logic of looping between player turns. It does not encompass the internal logic of executing a turn.

During the Game.run() loop, the list of players is looped through continuously, until a win condition is triggered to end the loop. For each player, a new turn instance is created. The logic sequence passes from Turn.playTurn() → Player.playerTurn() → GodCard.executeTurn(), which is analogous to a player being told it is their turn, and using their god card during the turn.



Demeter Build

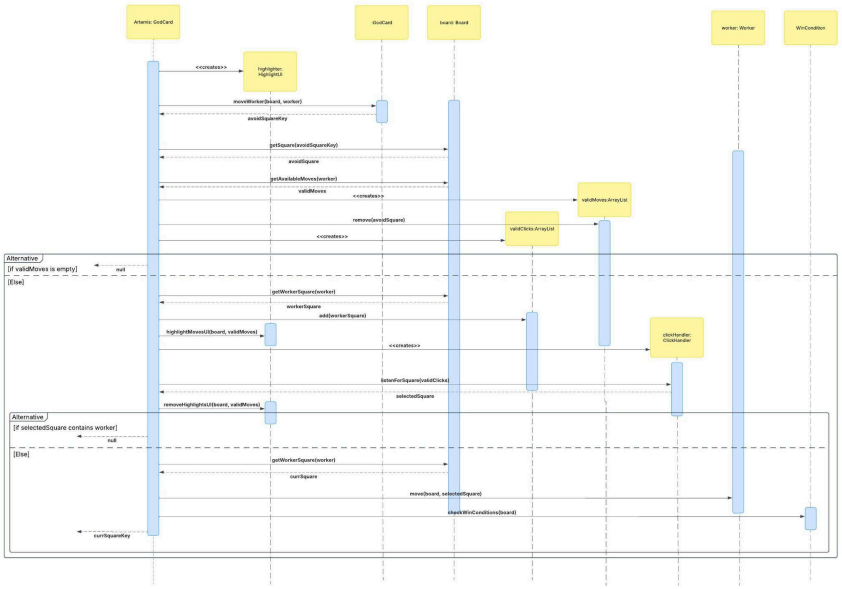
This sequence diagram encompasses the allowance of a second build for a player with the Demeter god card. It assumes that the player has already conducted the move section of their turn and buildBlock() has been called.



Demeter calls super.buildBlock() to perform the first build and retrieve the square that the player is not allowed to build on again. It then filters that square from the available builds, highlights the remaining squares using HighlighUI, and uses ClickHandler to listen for the player's second selection. If a valid square is selected, Demter builds a new level at that location.

Artemis Move

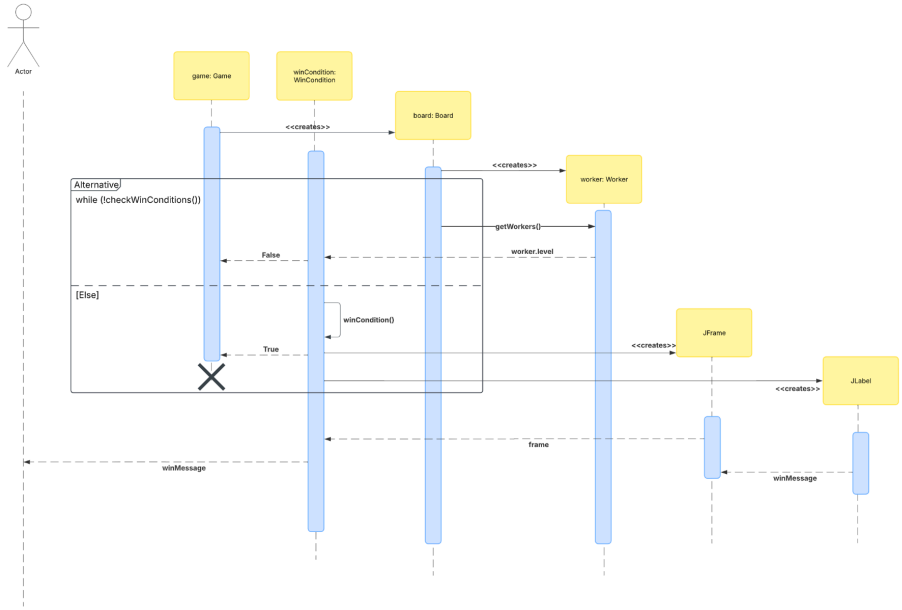
This sequence diagram is specific to demonstrating the allowance of a second move for a player with the Artemis god card. It assumes that the opponent has just finished their turn and moveWorker() has been called.



Artemis calls super.moveWorker() to perform the first move and retrieves the square to avoid for the second move. It then filters the second valid moves, highlights them using HighlightUI, and uses ClickHandler to listen for the player's second selection. If a valid square is selected, Artemis moves the worker again and checks for a win condition.

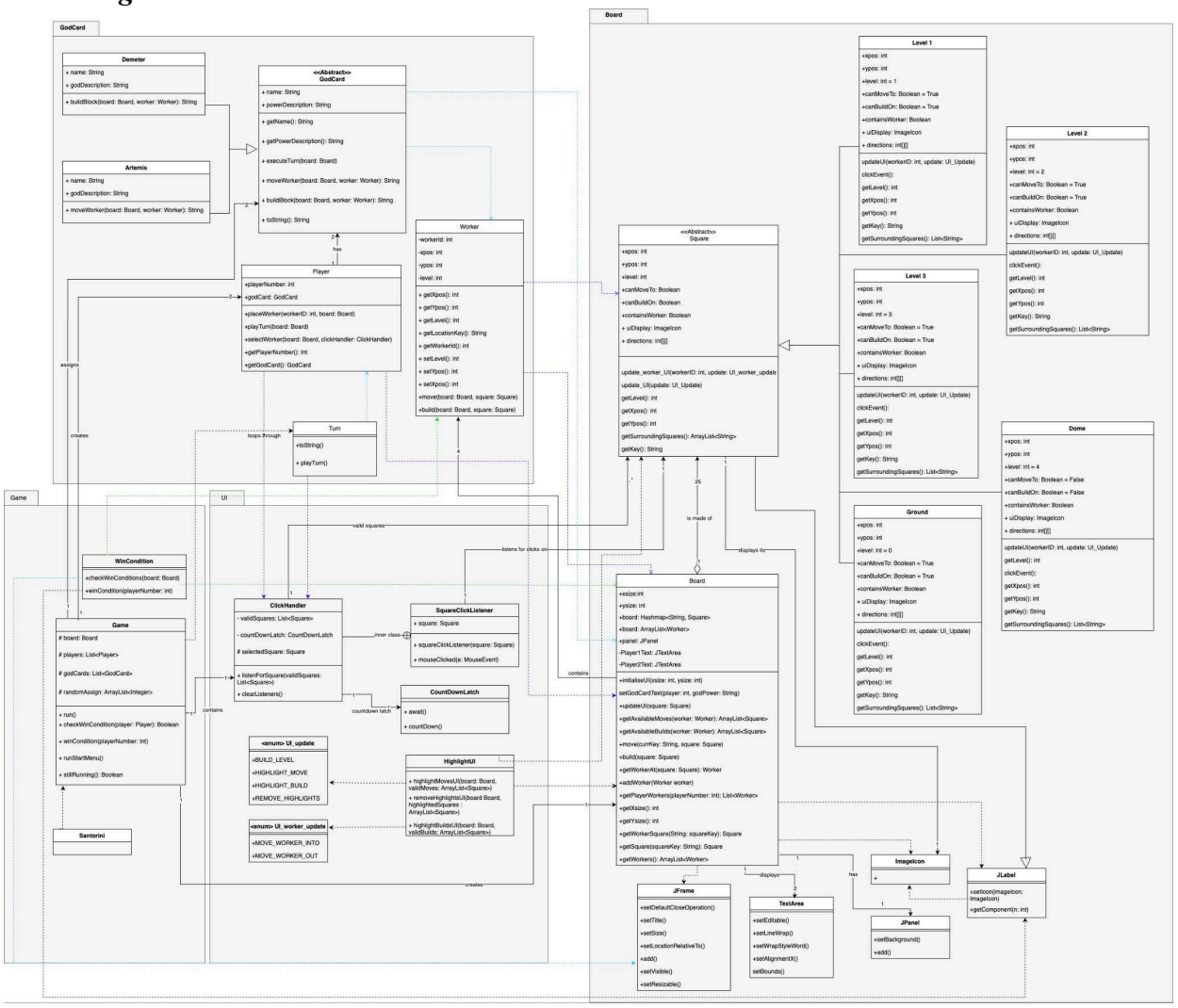
Win Condition

This sequence diagram only encompasses the Win Condition of when a player ascends to level 3 in basic gameplay, and does not encompass getting trapped, or God Card implications.



During the game loop that runs while the Win Conditions check is false, the worker's levels in the game are continuously checked, with the boolean checkWinCondition() method. If that method returns True, the winCondition() method is recursively called, and ends the game. This then additionally creates a JFrame, with a JLabel on it, informing the Actor (the Players) that the game has ended, and the winner.

UML Diagram



Key Classes

Click Handler

The Click Handler class was introduced after several iterations of our code. Initially it existed as a method within each Square, however this led to difficulties:

- Managing which squares were clickable on each turn: initialising all Squares with an inbuilt click handler makes all squares clickable at all times which violates game rules.
- Allowing different types of clicks to have different UI (e.g., highlight builds, highlight moves).
- Click handlers not unique to child classes of square: a separate type of click handler is also needed for worker selection.

Addressing these issues, implementing this as a method in Player class and Square classes technically worked and was our initial approach, however lead to a lot of repeated code. Thus, ClickHandler was extracted to be its own key class which can be instantiated and called wherever needed. This made the program more object oriented, as this ensured that the team took a DRY approach to programming.

Board

Another key class we included was Board which we decided was necessary to hold the current game state through information about a collection of Squares and how they interact with each other, rather than having this logic as methods within each individual Square. Key reasons for this included:

- Only need to initialise a singular board in the Game class, rather than having to initialise 25 different squares. Allows for flexibility in future designs where the Board composition might change, which can be executed without having to alter any lower level Square classes.
- Adheres to single responsibility principle: without a Board class, each Square would not only have to manage its position and UI, it would also have to be responsible for storing move/build validation and communication with each other Square, turning Square into a 'God class'. Instead, these extra responsibilities are extracted to the Board, reducing dependencies.
- Allows for ease of UI updates - Board knows where each of its squares is, and is responsible for delegating UI updates to each of them when necessary. This is also streamlined since the Board assigns each square a key, allowing it to be quickly accessed, rather than the Game having to parse through all squares.

WinCondition

The Win Condition class didn't exist in early iterations of our code, and was instead implemented as methods inside the Game class. The reasoning for this was that the team had interpreted the requirements to only check for win conditions at the start of the turn and at the end of the turn, thus meaning the win methods would only need to exist in the class (Game) where the looping turn logic was being implemented. However, further development led the team to realise that default win conditions could occur mid-turn, for example, when a player selects a trapped worker and due to the 'touch rule' (once a worker is selected, it must be used) they are unable to complete their turn, and thus lose the game. There were also issues discovered due to the win condition not being checked within a turn, in that, a worker could ascend to level 3 on the 'move' action of their turn, but this would not be declared as a win until they also completed the 'build' action of their turn. For these reasons, the win condition methods from Game were extracted to their own class, WinCondition, so that these checks could be performed across different sections of the game, without the need for repetitive code.

Key Relationships

Board to Square

The relationship between Board and Square is an aggregation, where Board maintains a collection of Square objects in a Hashmap. Although, Board does control aspects of the Square lifecycle through the build() method (which replaces existing Squares with new ones of higher levels), it is still considered aggregation rather than composition. This is due to the fact that Square objects are UI components that can exist beyond just being part of Board.

This aggregation relationship was chosen over composition because it allows Squares to function as distinct UI elements that can be manipulated and replaced during gameplay. The Board acts as a container and manager for Squares rather than their sole owner, enabling the dynamic level progression required by the

game mechanic. This approach allows each Square to manage its own visual representation while the Board manages their collective arrangement and game state progression.

GodCard to Worker

The relationship between a God Card and the two workers it applies to is a one-way associative relationship, where the God Card knows about the Worker. This allows the God Card to control the Worker. As God Card is an abstract class, the God Cards that extend this class (Artemis, Demeter), are therefore able to control the workers move and build operations.

This associative relationship was chosen over having Workers extend the God Card class (generalisation), as this ensures that players are able to play without the God Card behaviour, and abstracts the behaviour to individual God Card classes.

Turn to Player

This relationship is a one-way dependency where Turn knows about Player but Player does not know about Turn. Although it makes more intuitive sense that the Player would know about its Turn, when adhering to OOP, having a Turn class responsible for telling the Player to execute their Turn reduces the number of responsibilities that the Player class has. If instead, the relationship was the other way, the design becomes less maintainable, since any changes to a Player's logic may break the game's Turn mechanism, causing it to loop or stall.

Inheritance Decisions

There were two key inheritance decisions within our design:

1. Artemis & Demeter inherit from God Card

GodCard provides the default logic for the key move and build components of a turn. As an abstract class, these moveWorker() and buildBlock() methods are able to be overridden by child classes (in the scope of this sprint; Artemis and Demeter) to implement different types of move and build. However, even if a child class does not override the default behaviour, or if a player chooses not to use their GodCard, the turn can still be executed with the base methods without breaking the code. Without this inheritance, each individual God class would need to ensure that it implemented the basic move and build methods, as well as the powered-up move and build, which would lead to repetitive code in all of these classes.

2. Ground, Levels 1-3, & Dome inherit from Square

Since most of the squares share similar behaviour (can be built with the exception of Ground, and can be moved to with the exception of Dome), it made sense to extract this common behaviour into a parent class Square, allowing for inheritance. We created the Square class as it provided for a baseline functionality of the squares on the board, reducing redundancy in code as the number of repeated methods was very low. This increases the extensibility of the design as there is a consistent and clear approach to creating new types of squares, so in future extensions if a new type of square was introduced (e.g., Hole), instead of needing to construct its behaviour from scratch, it can simply extend Square, and adapt its constructor with appropriate booleans for canMoveTo (false) and canBuildOn (false), and uiDisplay Image.

Cardinalities

Board to Square 1..25

The aggregate relationship between Board and Square with cardinality 1..25, is defined as this because there is only one board during the game, and this board is comprised of 25 squares (5x5 dimension board). This relationship is defined for the current game requirements of having unchanging dimensions for the board. This cardinality was arrived at in response to the game rules, and the basic game board set up. Given that there is always a game board, from initialising the game to a win condition, and the number of squares comprising this board are unchanging, the cardinality is as so.

Turn to Player 1..*

The dependency between Turn and Player has a cardinality corresponding to '1 to many Turns for 1 Player'. Initially, the minimal cardinality of Turn was 0 as we considered the user story where a game could be restarted at any point. However, since this is out of the scope of this sprint, this cardinality was changed to 1, as each player must play at least once and continue to play until a win condition is reached. In practice,

more than 1 move is logically required to reach a win condition, but due to the unpredictability of the gameplay and player strategy, this is kept at 1 as a safe standard.

Design Patterns

Template Method

To incorporate god card powers, we employed the Template Method design pattern to structure how a player's turn is executed. The game enforces a consistent rule that each turn must follow the sequence: move first, then build. This invariant made the Template Method an ideal fit. We defined a template executeTurn() method in the abstract GodCard class, which outlines the turn structure by sequentially calling moveWorker() and buildBlock(). These two methods have default implementations, but can be selectively overridden by concrete subclasses to introduce custom behaviour. This pattern allowed us to encapsulate the common structure of a turn while supporting extensibility for the unique powers of each God card. For example, Artemis overrides only the moveWorker() method to allow moving twice (without returning to the original position), while still relying on the default buildBlock() logic. This modular approach avoids code duplication and keeps the core game loop consistent.

We initially considered the Strategy pattern, as documented in Draft 4 of our UML. In this design, each GodCard would act as a strategy implementing a GodCardStrategy interface, with methods like modifyMoves() and modifyBuilds() to influence available actions. While this would have allowed for flexible customisation of behaviour, it did not naturally enforce the strict "move then build" structure of a turn. Each strategy would have needed extra logic to ensure that the sequence of actions was preserved, introducing unnecessary complexity and the risk of violating game rules. In contrast, the Template Method guarantees the correct turn order while still supporting behavioural variation through method overriding. Ultimately, the Template Method pattern aligned better with the fixed structure and partial customization requirements of Santorini turns, offering a cleaner and more robust solution.

Testing

Key: *Green* indicates that a test passed, *Red* indicates that a test failed, and *Orange* indicates that a test initially failed but was later passed.

Worker Placement

Testing of the randomised worker placement logic involved initialising various game boards (all squares contain workers already, some squares contain workers already, board does not contain any workers) and running the game to determine if the correct logic is followed for those situations. This testing method was also aided by the use of terminal print statements.

1. *Worker cannot be placed : input is a board initialised with all squares containsWorker = true.*

Expected output: No workers are able to be placed.
Actual output: No workers are able to be placed and the game enters an infinite loop searching for placement positions.

2. *First randomly generated position is occupied: input is board initialised with the first 4x5 of the squares set to containWorker = true*

Expected output: 4 series of print statements:“Searching for position” and 1 print statement:“Found”
Actual output: 4 series of print statements: “Searching for position” and 1 print statement: “Found”

3. *Worker is placed in an available position : input is board initialised with the first 4x5 of the squares set to containWorker = true*

Expected output: All 4 workers will be placed within the final column (5).
Actual output: all 4 workers are placed in the final column (5) and the win condition is triggered if all 4 workers are placed with no gap in between.

1. *Randomised Worker placement*

Expected output: When the board is initialised, four of the squares are occupied by workers (two of each team), and if closed and reopened they are initialised to different squares.

Actual output: When the game opens a number of times in a row, all four workers appear every time, in random positions each run.

Select worker/Click handling

Select worker was the functionality that required UI interaction and subsequent click handling. Testing strategies for this involved printing statements and running the game to check if listeners were being created and if clicks were being handled correctly

1. *Click listeners being created for the square of a worker at 3,1*

Expected output: print statement: "new listener for: 3, 1" and when you hover over the 3,1 square, the mouse UI should change

Actual output: print statement: "new listener for: 3, 1", but hovering over 3,1 worker square, the mouse UI does not change. It changes at square 1,3.

2. *Registering a click on a click listener at the square of a worker at 3,1*

Expected output: print statement: "clicked!"

Actual output: print statement: "clicked!"

3. *Click and store worker at square 3,1*

Expected output: print statement: "selected worker: 3,1", where "3,1" string comes from worker.getLocationKey()

Actual output: print statement: "selected worker: 3,1"

4. *Click any other tile other than player's workers to ensure only player's workers are selectable*

Expected output: nothing

Actual output: nothing

User Interface

Testing if the User Interface displays the board correctly, with all square designs (e.g. worker, ground, level++, dome, move/build highlights) centred appropriately and scaled similarly. The testing strategy used for this aspect was to play the game, testing a number of different scenarios outlined below and looking to see if the expected output was achieved.

2. *Board initialisation*

Expected output: When the main function of Santorini is run, a pop-up appears, with a game board full of 25 level 0 squares (Ground).

Actual output: When the main function of Santorini is run, a pop-up appears, with a game board full of 25 level 0 squares (Ground).

Move/Build

Testing strategies to move and build functionalities involved running the game and checking if the correct UI would display as well as printing statements to confirm that the board model was being updated correctly too.

1. *Clicking on a worker to move them to another square*

Expected output: All possible destination squares a worker can move to are highlighted, with the design fully visible and centred in the panel.

Actual output: All possible destination squares a worker can move to are highlighted, with the design fully visible and centred in the panel.

2. *Click an available move at 3,1 to complete a move*

Expected output: 2 print statements: "worker current location: 3,2" then "worker new location: 3,1" and UI updated to show worker on new square

Actual output: 2 print statements: "worker current location: 3,2" then "worker new location: 3,1" and UI updated to show worker on new square

3. *Looking for available builds after completing a worker's move*

Expected output: All possible destination squares a worker can build on are highlighted, with the design fully visible and centred in the panel.

Actual output: All possible destination squares a worker can build on are highlighted, with the design fully visible and centred in the panel.

4. *Building a Level or a Dome*

Expected output: All levels and domes on screen are sized the same, and are all visible and distinguishable
Actual output: All levels and domes on screen are sized the same, and are all visible and distinguishable

5. *Player standing on Ground and all Levels*

Expected output: Player UI updates to the same Player UI when they change squares, and changes to the correct level
Actual output: Player UI updates to the same Player UI when they change squares, and changes to the correct level

Win Condition

Testing if the Win Condition is correctly triggered when any event on the board occurs that should trigger a win. The testing strategy used for this aspect was to play the game, testing a number of different scenarios outlined below and looking to see if the expected output was achieved.

1. *Using the input of gameplay that ends when Artemis reaches level 3 after first move:*

Expected output: Pop up message declaring that Artemis's Player wins
Actual output: Win message pop up appears on top of the board to indicate the Player that won.

2. *Using the input of gameplay that ends when Demeter reaches level 3 after move:*

Expected output: Pop up message declaring that Demeter's Player wins
Actual output: Win message pop up appears on top of the board to indicate the Player that won.

3. *Using the input of gameplay that ends when Player 1 reaches level 3:*

Expected output: Pop up message declaring that Player 1 wins
Actual output: Pop up message declaring that Player 1 wins

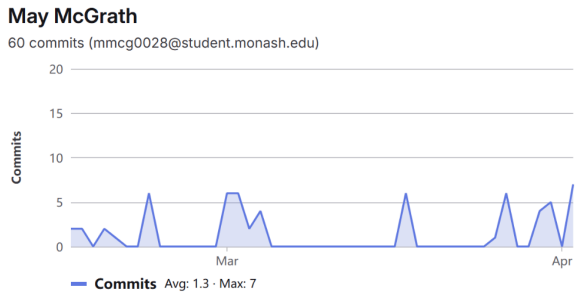
4. *Using the input of gameplay that ends when both of Player 1's workers have no possible moves:*

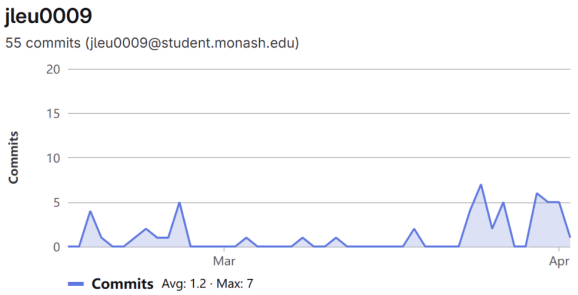
Expected output: Pop up message declaring that Player 2 wins
Actual output: Pop up message declaring that Player 2 wins

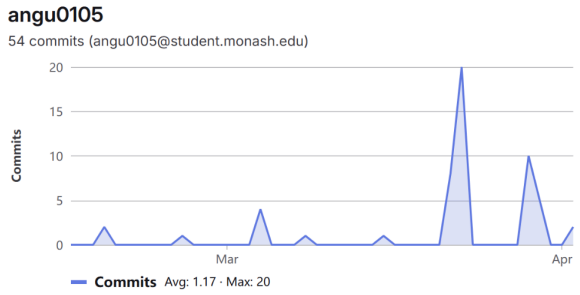
5. *Using the input of gameplay where one of Player 1's workers are trapped:*

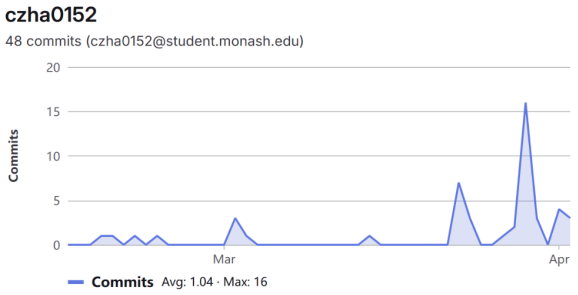
Expected output: Gameplay continues using the free worker.
Actual output: Gameplay continues as long as the trapped worker is not clicked, if clicked, the game board freezes. This is considered as a pass, as the assumption is made that a player will not select a trapped worker if their other worker is free.

Contributor Analytics









Creating the Executable

Instructions to create the executable can be found here in the team git: [Executable Instructions]